Bryant Har

Optimization Strategies

Very broadly, I had three main versions of my BigNum library. Here they are enumerated:

Version 1

The first version was based on my ex-nihilo assumptions about the project and what the major considerations would be. Specifically, I had a couple preconceptions about how to best approach and represent the numbers. It was pretty clear from the onset that this would be used for RSA purposes, but I assumed that the numbers probably wouldn't be very large. I was expecting something like 256 bit RSA, requiring no more than a small segmented array. Here were a list of some early thoughts

- Since the text was being streamed in, the overhead of string parsing may be nontrivial. As such, I decided to keep the internal representation as a string initially, as I assumed the slowdown of dealing with chars wouldn't be major enough to switch to fixed-width arrays with machine word-size segments.
- Encrypt is very fast since the exponent is small.
    - It's probably not worth optimizing compared to decrypt.
- Everything else, at least for a version zero, could be done with simple grade school arithmetic operations.
- Modular exponentiation, of course, could be done by the fast exponentiation method with mod identities.
- Freely resizable strings for storing internal representations of the numbers. For version 1, there isn't much of a need to think about word sizes or memory usage.

At this point, I was pretty happy with the bare working solution. It passed on autograder interestingly, although I don't have the benchmark. I hadn't timed it, but anecdotally, `The Road Not Taken` poem didn't take longer than a water break.

There wasn't a real need to check the core dumps or gprof anything either, as there were a lot of low hanging fruit optimizations.

Version 2 (~10x speedup)

For version 2, the natural next step was simply to multithread it. This was a fairly simple choice, as the string streaming in wasn't a very expensive operation. The main bottleneck was obviously

the decryption which was extremely parallelizable. Here were my primary observations when making this:

- A very simple model to thread this task would be to make a simple threadpool architecture with workers and a pre-allocated memory buffer to store the intermediate results.
    - The only issue is pre-empting how large this reserve size would be. I just had an exponential resizing scheme. While one resize would be optimal, resizing as lines get streamed in isn't a major cost.
- Each line was also partitionable into an upper and lower line. We can decrypt these separately and add it onto the task queue for workers to pick up. Those are defined as decrypt_upper_line and decrypt_lower_line.
- We can also constexpr the ingestion of constants.
- Ascii conversion was also highly parallelizable and a good candidate for SIMD. This wasn't a major cost either though.
- The number of threads were defined using thread::hardware_concurrency(), though I suspect this is unreliable. A more reliable solution would be to set a #DEFINE directive in the CMakeList.

```
cat one-liner.txt  0.00s user 0.00s system 52% cpu 0.002 total
bignum e  0.24s user 0.00s system 90% cpu 0.266 total
bignum d  8.79s user 0.04s system 681% cpu 1.296 total
```

This version was fairly solid. For the `The Road Not Taken` poem, it took around 9s locally, or around 5247ms on autograder.

This was also very promising, as my CPU usage almost hit 700%, which indicates that my threadpool implementation did a very solid job at multithreading decrypt.

*Version 3 (~**100x speedup)***

Here is where the more heavy duty optimizations came into play. At this point, I was still using strings as my internal representation. It was easy to parse and ingest from the input stream, but slow to work with due to having to shift the character offset. Here's a sample of the gprof output I observed. A few key notes follow.

```
Flat profile:

Each sample counts as 0.01 seconds.
  %   cumulative   self              self     total
 time   seconds   seconds    calls  s/call   s/call  name
 21.98      2.13     2.13   367274     0.00     0.00  BigNum::subtractStrings(std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > const&, std::__cxx11::basic_string<char, std::
char_traits<char>, std::allocator<char> > const&)
 11.56      3.25     1.12   367386     0.00     0.00  BigNum::addStrings(std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > const&, std::__cxx11::basic_string<char, std::char_
traits<char>, std::allocator<char> > const&)
 10.01      4.22     0.97 292759949     0.00     0.00  std::remove_reference<char&>::type&& std::move<char&>(char&)
  7.64      4.96     0.74 192605242     0.00     0.00  __gnu_cxx::__normal_iterator<char*, std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > >::base() const
  7.43      5.68     0.72 192016161     0.00     0.00  __gnu_cxx::__normal_iterator<char*, std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > >::operator*() const
  7.02      6.36     0.68 96191350     0.00     0.00  std::enable_if<std::__and_<std::__not_<std::__is_tuple_like<char> >, std::is_move_constructible<char>, std::is_move_assignable<char> >::value, void>::
type std::swap<char>(char&, char&)
  5.78      6.92     0.56                             _init
  5.37      7.44     0.52 94241923     0.00     0.00  void std::iter_swap<__gnu_cxx::__normal_iterator<char*, std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > >, __gnu_cxx::
__normal_iterator<char*, std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > > >(__gnu_cxx::__normal_iterator<char*, std::__cxx11::basic_string<char, std::char_traits<char>, st
d::allocator<char> > >, __gnu_cxx::__normal_iterator<char*, std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > >)
  5.16      7.94     0.50 93445822     0.00     0.00  bool __gnu_cxx::operator< <char*, std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > >(__gnu_cxx::__normal_iterator<char*
, std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > > const&, __gnu_cxx::__normal_iterator<char*, std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char
> > > const&)
  5.06      8.43     0.49 98416181     0.00     0.00  __gnu_cxx::__normal_iterator<char*, std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > >::operator++()
  4.59      8.88     0.45 99095959     0.00     0.00  __gnu_cxx::__normal_iterator<char*, std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > >::operator--()
  4.13      9.28     0.40  1263436     0.00     0.00  void std::__reverse<__gnu_cxx::__normal_iterator<char*, std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > >(__gnu_cxx
:__normal_iterator<char*, std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > >, __gnu_cxx::__normal_iterator<char*, std::__cxx11::basic_string<char, std::char_traits<char>, st
d::allocator<char> > >, std::random_access_iterator_tag)
  1.75      9.45     0.17      534     0.00     0.00  multiplyStringsLimit(std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > const&, std::__cxx11::basic_string<char, std::cha
r_traits<char>, std::allocator<char> > const&, unsigned long)
  1.24      9.56     0.12                             bool __gnu_cxx::operator!=<char*, std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > >(__gnu_cxx::__normal_iterator<char*
, std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > > const&, __gnu_cxx::__normal_iterator<char*, std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char
> > > const&)
  0.57      9.62     0.06       12     0.00     0.00  std::remove_reference<int (*&)(int) noexcept>::type&& std::move<int (*&)(int) noexcept>(int (*&)(int) noexcept)
  0.41      9.66     0.04      901     0.00     0.00  BigNum::multiplyStrings(std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > const&, std::__cxx11::basic_string<char, std::
char_traits<char>, std::allocator<char> > const&)
  0.10      9.67     0.01  1228229     0.00     0.00  bool __gnu_cxx::operator==<char*, std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > >(__gnu_cxx::__normal_iterator<char*
, std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > > const&, __gnu_cxx::__normal_iterator<char*, std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char
> > > const&)
  0.10      9.68     0.01  1206796     0.00     0.00  void std::reverse<__gnu_cxx::__normal_iterator<char*, std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > >(__gnu_cxx::_
_normal_iterator<char*, std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > >, __gnu_cxx::__normal_iterator<char*, std::__cxx11::basic_string<char, std::char_traits<char>, std:
:allocator<char> > >)
  0.10      9.69     0.01   826483     0.00     0.00  BigNum::compareStrings(std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > const&, std::__cxx11::basic_string<char, std::c
har_traits<char>, std::allocator<char> > const&)
  0.00      9.69     0.00  1219960     0.00     0.00  std::iterator_traits<__gnu_cxx::__normal_iterator<char*, std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > >::iterator
_category std::__iterator_category<__gnu_cxx::__normal_iterator<char*, std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > > >(__gnu_cxx::__normal_iterator<char*, std::__cxx11::
:basic_string<char, std::char_traits<char>, std::allocator<char> > > const&)
  0.00      9.69     0.00    80891     0.00     0.00  bool std::operator>(char, std::char_traits<char>, std::allocator<char> >(std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char>
 > const&, std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > const&)
  0.00      9.69     0.00    79705     0.00     0.00  bool std::operator< <char, std::char_traits<char>, std::allocator<char> >(std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char
> > const&, std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > const&)
  0.00      9.69     0.00     4538     0.00     0.00  bool std::operator==<char, std::char_traits<char>, std::allocator<char> >(std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char
> > const&, char const*)
  0.00      9.69     0.00     3554     0.00     0.00  std::iterator_traits<char const*>::difference_type std::__distance<char const*>(char const*, char const*, std::random_access_iterator_tag)
  0.00      9.69     0.00     3515     0.00     0.00  std::iterator_traits<char const*>::iterator_category std::__iterator_category<char const*>(char const* const&)
  0.00      9.69     0.00     3505     0.00     0.00  std::iterator_traits<char const*>::difference_type std::distance<char const*>(char const*, char const*)
  0.00      9.69     0.00     3325     0.00     0.00  bool __gnu_cxx::__is_null_pointer<char const>(char const*)
  0.00      9.69     0.00     3058     0.00     0.00  std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> >::basic_string<std::allocator<char> >(unsigned long, char, std::allocat
.
```

Version 2 - GProf Flat Profile

Noteworthy things about this gprof by looking at the top heavy hitter functions:

- Division is the heaviest hitter. This makes sense as it uses a combination of modulus/multiplication and other subcalls. It's expensive relative to these other ops as a result.
    - It's also used heavily in modular exponentiation, as it's a core part of the modulus operation. Modulus is comparable to integer division.
- Lots of time spent on character operations and **swap/move operations** (most of the templated code). This was due to the heavy usage of reverse() operations and memory swaps/moves within the same memory segment.
    - Endianness is a big issue here. Strings are naturally stored in big endian, though little endian would reduce the need for reversals when working additively left to right.
    - **Swap operations grow expensive, because I store the data in big endian.**
- Overhead of character conversion and string parse methods add up

    - I expected this to be comparable to a simple vector of numbers, but this seems untrue. The time overhead of these char operations is prohibitive and very nontrivial.
- Heavy use of calls to add and subtract.
    - This is generally unavoidable and conducted through subcalls from multiply/modulus/division.
- Compared to vectors, arrays exploit caches better and work well with SIMD, loop unrolling, and constexpr (at least until Cpp20). Also store digits using quadwords.

Optimization Strategy

These are separate from the 6 optimizations specified in the handout, most of which I implemented. The major takeaway from the gprof was that my string internal data format was very detrimental. A few optimizations came to mind. Strategies generally are of two archetypes.

Memory/Data Optimizations

Data Representation

From gprof, I noticed that the string internal representation was hurting my performance, so I refactored all of my algorithms to use a different format. These included:

- 64 bit uints as my standard number format
    - Modern x86 processors have registers that support quadwords
- **Little endian over big endian (reduce # of swap/move operations)**
    - Does not require digit reversal to do some bitwise operations (subtraction, addition).
    - Slightly hard to conduct bitshifts with, but array bitshifts are fast with SIMD. Bitshifts are generally very amenable to SIMD.
- No power of 2 radix
    - Considered it, as it would make bitshifts very fast, but ultimately chose base 1e9 since that way you don't incur the overhead of converting a base 10 number to some power of 2 radix.
    - Base 10 to base 1e9 is very fast and also parallelizable.
- Base 1e9
    - 64 bit uints support up to ~1e19 digits.
    - A 1e9 workspace makes sense, as I expect to multiply segments
- 36-digit Fixed-width segment arrays instead of vector (Arrays over vectors)
    - Cpp arrays are more easy to SIMD and before C++20 was ***much*** easier to constexpr.

    - Constexpr is important since we can easily convert our constants beforehand.
    - 36 digit was chosen as 36 * 9=324, which can accommodate the product of two 154 digit numbers if necessary.
    - The "effective workspace" was then 18 digits with radix 1e9, or one 154 digit number in base 10.

Operational Changes

- Instead of halving (expensive division), you can scale the number and conduct a bitshift, which is cheap and friendly to SIMD. Multiply by 5 in base 10 or 500 million in base 1e9.
- Binary search the single-digit by full number product, as otherwise you would have to do a linear scan through all digits from 0 to 1e9.
    - ***This would also be very impractical to cache in 1e9 radix***
- Pass everything by reference and inline most functions to avoid function call overheads (very minor).
- Constexpr most of the constants.
- Set a "workspace size" of 154 digits, as that is the modulus. Stop computing operations if they exceed the modulus ("workspace size"). Conduct the modulus early if possible.

Algorithmic Optimizations

The primary algorithms are drawn from the following reference, which I've already been fairy familiar with in the past.

Handbook of Applied Cryptography - https://cacr.uwaterloo.ca/hac/about/chap14.pdf

Fast Modular Exponentiation

This is pretty standard. The fast exponentiation method (convert exponent to a bit string) is pretty common already. I had this implemented from the get-go, though the parts of modular exponentiation can be optimized by below methods. Additionally, halving operations can be replaced in base 10 by scaling by 5 and executing a right bitshift (which is easily to pair with SIMD)

The book also suggests we may be able to jump/skip through some exponentiation using edge case CRT/Garner's. I'm not very familiar with these methods though. The major optimization is barrett's reduction, which I actually am familiar with though didn't implement.

Multiplication

Karatsuba's is the most simple way to optimize multiplication operations.

- From my experience (no theoretical reasoning), past 32 digits, it's typically faster than grade school methods. Asymptotically, it's the standard O(n^log(3)) or O(n^1.58).

Other methods like FFT have very high overheads that I don't expect to fare better in k=154 digit operations.

Didn't bother doing this as my intermediate result was already fast enough to my liking.

Division/Modulus (main operation on critical path)

- The main optimizations here were using montgomery form or barrett reductions (I've used barrett before).
    - Our requirements very easily fit the Barrett reduction needs (i.e. value under modulo no more than twice the modulus number n).
    - Back of the envelope reasoning suggests it'll yield a 10-100x speedup since division is the heaviest hitter.
    - This was basically overkill though

Result:

Using the new data format and changing the endianness, I was able to achieve ~100x speedup from the previous version. The resultant performance was (~130ms local, 191ms autograder)

I didn't follow through with barrett reduction, as it was already fast enough, but back of the envelope suggests that implementing some division optimizations and barrett would yield a 10x speedup, yielding the ~10ms performance on the top of the leaderboard. I've spoken to Jeffrey Q and yeah he affirmed my suspicions that that's basically what he did to get from 300ms -> 9ms.

Further, swapping in karatsubas and doing some minor caching would improve that further.

```
-- Build files have been written to: /home/bjh254/hw4v2/hw4 v2
Consolidate compiler generated dependencies of target bignum
[ 25%] Building CXX object CMakeFiles/bignum.dir/main.cpp.o
[ 50%] Building CXX object CMakeFiles/bignum.dir/threadpool.cpp.o
[ 75%] Building CXX object CMakeFiles/bignum.dir/bignum.cpp.o
[100%] Linking CXX executable bignum
[100%] Built target bignum
(reverse-i-search)`bignum': ^Cgnum \*
bjh254@en-ci-cisugcl17:~/hw4v2/hw4 v2$ time cat one-liner.txt | bignum e | bignum d
Two roads diverged in a yellow wood,
And sorry I could not travel both
And be one traveler, long I stood
And looked down one as far as I could
To where it bent in the undergrowth;

real    0m0.133s
user    0m0.453s
sys     0m0.003s
bjh254@en-ci-cisugcl17:~/hw4v2/hw4 v2$
```

Local Timing (V3)

Re-analyzing Gprof

Re-analyzing gprof, there aren't too many surprises.

Notes from gprof (below):

- As expected, division/modulus are still the most expensive operations on the critical path, though that's to be expected.
    - These are called primarily through decryptUpperLine, decryptLowerLine
    - Encrypt is very fast since the exponent is small.
- Some of the operations conducted may still benefit from explicitly specifying SIMD. For example the bitshift and the multiplication operation should be partially or fully SIMD-able.
- **Importantly, there aren't as many swap operations and move operations as before when I used strings. This is a critical speedup due to the change in data format/endianness.**

```
Flat profile:

Each sample counts as 0.01 seconds.
  %   cumulative   self              self     total
 time   seconds   seconds    calls  ms/call  ms/call  name
 97.50      0.39     0.39     7598     0.05     0.05  BigNum::divideNums(std::array<unsigned long, 36ul> const&, std::array<unsigned long, 36ul> const&)
  2.50      0.40     0.01    12658     0.00     0.00  BigNum::multiplyNums(std::array<unsigned long, 36ul> const&, std::array<unsigned long, 36ul> const&)
  0.00      0.40     0.00     7591     0.00     0.05  BigNum::modulusNums(std::array<unsigned long, 36ul> const&, std::array<unsigned long, 36ul> const&)
  0.00      0.40     0.00       11     0.00     0.00  BigNum::BigNum(std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > const&)
  0.00      0.40     0.00       10     0.00    40.00  BigNum::modExpNums(std::array<unsigned long, 36ul> const&, std::array<unsigned long, 36ul> const&, std::array<unsigned long, 36ul> const&)
  0.00      0.40     0.00       10     0.00    40.00  BigNum::decrypt[abi:cxx11]() const
  0.00      0.40     0.00        5     0.00    40.00  decryptLowerLine(std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > const&)
  0.00      0.40     0.00        5     0.00    40.00  decryptUpperLine(std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > const&)
  0.00      0.40     0.00        5     0.00     0.00  std::_Function_handler<void (), decrypt()::{lambda()#2}>::_M_manager(std::_Any_data&, std::_Any_data const&, std::_Manager_operation)
  0.00      0.40     0.00        5     0.00    40.00  std::_Function_handler<void (), decrypt()::{lambda()#2}>::_M_invoke(std::_Any_data const&)
  0.00      0.40     0.00        5     0.00     0.00  std::_Function_handler<void (), decrypt()::{lambda()#1}>::_M_manager(std::_Any_data&, std::_Any_data const&, std::_Manager_operation)
  0.00      0.40     0.00        5     0.00    40.00  std::_Function_handler<void (), decrypt()::{lambda()#1}>::_M_invoke(std::_Any_data const&)
  0.00      0.40     0.00        3     0.00     0.00  std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> >::basic_string<std::allocator<char> >(char const*, std::alloca
tor<char> const&) [clone .constprop.1]
  0.00      0.40     0.00        2     0.00     0.00  std::vector<std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> >, std::allocator<std::__cxx11::basic_string<char,
std::char_traits<char>, std::allocator<char> > > >::_M_default_append(unsigned long)
  0.00      0.40     0.00        1     0.00     0.00  decrypt()
  0.00      0.40     0.00        1     0.00     0.00  ThreadPool::finish()
  0.00      0.40     0.00        1     0.00     0.00  ThreadPool::ThreadPool(unsigned long)
  0.00      0.40     0.00        1     0.00     0.00  ThreadPool::~ThreadPool()
```

Gprof Flat Profile (V3)

```
                     Call graph (explanation follows)


granularity: each sample hit covers 4 byte(s) for 2.50% of 0.40 seconds

index % time    self  children    called     name
                0.00    0.40      10/10          BigNum::decrypt[abi:cxx11]() const [2]
[1]    100.0    0.00    0.40      10         BigNum::modExpNums(std::array<unsigned long, 36ul> const&, std::array<unsigned long, 36ul> const&, std::array<unsigned long, 36ul> const&) [1]
                0.00    0.39    7591/7591        BigNum::modulusNums(std::array<unsigned long, 36ul> const&, std::array<unsigned long, 36ul> const&) [5]
                0.01    0.00   12658/12658       BigNum::multiplyNums(std::array<unsigned long, 36ul> const&, std::array<unsigned long, 36ul> const&) [10]
-----------------------------------------------
                0.00    0.20       5/10          decryptUpperLine(std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > const&) [7]
                0.00    0.20       5/10          decryptLowerLine(std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > const&) [6]
[2]    100.0    0.00    0.40      10         BigNum::decrypt[abi:cxx11]() const [2]
                0.00    0.40      10/10          BigNum::modExpNums(std::array<unsigned long, 36ul> const&, std::array<unsigned long, 36ul> const&, std::array<unsigned long, 36ul> const&) [1]
-----------------------------------------------
                                               <spontaneous>
[3]    100.0    0.00    0.40                std::thread::_State_impl<std::thread::_Invoker<std::tuple<ThreadPool::ThreadPool(unsigned long)::{lambda()#1}> > >::_M_run() [3]
                0.00    0.20       5/5           std::_Function_handler<void (), decrypt()::{lambda()#2}>::_M_invoke(std::_Any_data const&) [8]
                0.00    0.20       5/5           std::_Function_handler<void (), decrypt()::{lambda()#1}>::_M_invoke(std::_Any_data const&) [9]
                0.00    0.00       5/5           std::_Function_handler<void (), decrypt()::{lambda()#1}>::_M_manager(std::_Any_data&, std::_Any_data const&, std::_Manager_operation) [20]
                0.00    0.00       5/5           std::_Function_handler<void (), decrypt()::{lambda()#2}>::_M_manager(std::_Any_data&, std::_Any_data const&, std::_Manager_operation) [19]
-----------------------------------------------
                0.39    0.00    7598/7598        BigNum::modulusNums(std::array<unsigned long, 36ul> const&, std::array<unsigned long, 36ul> const&) [5]
[4]     97.5    0.39    0.00    7598         BigNum::divideNums(std::array<unsigned long, 36ul> const&, std::array<unsigned long, 36ul> const&) [4]
-----------------------------------------------
                0.00    0.39    7591/7591        BigNum::modExpNums(std::array<unsigned long, 36ul> const&, std::array<unsigned long, 36ul> const&, std::array<unsigned long, 36ul> const&) [1]
[5]     97.5    0.00    0.39    7591         BigNum::modulusNums(std::array<unsigned long, 36ul> const&, std::array<unsigned long, 36ul> const&) [5]
                0.39    0.00    7598/7598        BigNum::divideNums(std::array<unsigned long, 36ul> const&, std::array<unsigned long, 36ul> const&) [4]
-----------------------------------------------
                0.00    0.20       5/5           std::_Function_handler<void (), decrypt()::{lambda()#1}>::_M_invoke(std::_Any_data const&) [9]
[6]     50.0    0.00    0.20       5         decryptLowerLine(std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > const&) [6]
                0.00    0.20       5/10          BigNum::decrypt[abi:cxx11]() const [2]
                0.00    0.00       5/11          BigNum::BigNum(std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > const&) [18]
```

Gprof Call Graph (V3)